Tuần 08-09: Thực hành Spring Security

1. Spring Security là gì?

Spring Security là framework bảo mật cho ứng dụng web Java, làm hai việc chính:

- **Authentication (xác thực):** bạn là ai? (đăng nhập)
- **Authorization (phân quyền):** bạn được làm gì?

Ví dụ đời thường: đi vào tòa nhà văn phòng.

- Bảo vệ kiểm tra thẻ nhân viên → xác thực.
- Thẻ của bạn chỉ mở được tầng 3, không vào được phòng server tầng 10 → phân quyền.

Framework hỗ trợ nhiều cơ chế đăng nhập:

Cơ chế	Giải thích	Ví dụ
HTTP Basic	Gửi username:password (mã hóa base64) trong header mỗi request. Chỉ an toàn khi dùng HTTPS	Gọi API nội bộ bằng Postman
LDAP	Xác thực qua kho người dùng tập trung của tổ chức	Đăng nhập bằng tài khoản Active Directory của công ty
OpenID / OAuth2	Đăng nhập qua bên thứ ba	"Login with Google"
JAAS	API xác thực chuẩn của Java	Tận dụng module JAAS có sẵn
Tùy biến	Tự viết logic qua UserDetailsService, AuthenticationProvider	Đăng nhập bằng dữ liệu trong MySQL

2. Các khái niệm cơ bản

Khái niệm	Ý nghĩa	Ví dụ
Principal	Người dùng đang thực hiện hành động	User nam@gmail.com đang đăng nhập
Authentication	Xác nhận thông tin đăng nhập đúng	Kiểm tra username + password khớp DB
Authorization	Quy định principal được truy cập gì	/admin/** chỉ dành cho ADMIN
GrantedAuthority	Quyền cụ thể được cấp cho principal	ROLE_ADMIN, ROLE_USER, can_delete_product

Ví dụ gộp lại: Nam đăng nhập thành công (authentication) → Nam là Principal → Nam có `ROLE_USER` (GrantedAuthority) → khi Nam vào `/admin`, hệ thống kiểm tra quyền (authorization) và từ chối vì Nam không có `ROLE_ADMIN`.

3. Các thành phần kiến trúc

- **SecurityContext:** "két sắt" chứa thông tin bảo mật của người dùng hiện tại, quan trọng nhất là đối tượng `Authentication`.
- **SecurityContextHolder:** lớp tiện ích để truy cập `SecurityContext` từ bất kỳ đâu. Mặc định dùng `ThreadLocal`, nên mỗi request (mỗi thread) có thông tin riêng, không bị lẫn giữa các người dùng.
- **AuthenticationManager:** "quản lý" điều phối việc xác thực. Nó không tự kiểm tra mà giao cho các `AuthenticationProvider`.
- **AuthenticationProvider:** "nhân viên" thực sự kiểm tra, biết lấy dữ liệu user từ nguồn nào (DB, LDAP...) và so khớp mật khẩu.

Ví dụ lấy thông tin user đang đăng nhập trong code:

```
Authentication auth = SecurityContextHolder.getContext().getAuthentication();
String username = auth.getName();                                    // "nam"
Collection<? extends GrantedAuthority> roles = auth.getAuthorities(); // [ROLE_USER]
```

4. Các đối tượng liên quan đến xác thực

- **Authentication object:** trước khi xác thực chứa thông tin thô (username/password người dùng nhập). Sau khi xác thực thành công chứa Principal + quyền hạn.
- **UserDetails:** "hồ sơ" người dùng gồm username, password (đã mã hóa), danh sách quyền, và trạng thái tài khoản (khóa, hết hạn, bị vô hiệu...).
- **UserDetailsService:** nhận username, tìm trong kho dữ liệu, trả về `UserDetails`.

Ví dụ tự triển khai `UserDetailsService` với database:

```
@Repository
public interface UserRepository  extend JpaRepository(User, Int){

}


@Service
public class MyUserDetailsService implements UserDetailsService {

    @Autowired
    private UserRepository userRepository;

    @Override
    public UserDetails loadUserByUsername(String username) throws UsernameNotFoundException {
        UserEntity user = userRepository.findByUsername(username)
                .orElseThrow(() -> new UsernameNotFoundException("Không tìm thấy: " + username));

        return org.springframework.security.core.userdetails.User
                .withUsername(user.getUsername())
                .password(user.getPassword())      // đã mã hóa BCrypt
                .roles(user.getRole())             // ví dụ "ADMIN" → ROLE_ADMIN
                .build();
    }
}
```

5. Luồng xử lý đăng nhập

```
HTTP Request
    ↓
AuthenticationFilter     → chặn request, lấy username/password, tạo token chưa xác thực
    ↓
AuthenticationManager    → điều phối, giao cho Provider
    ↓
AuthenticationProvider   → so sánh mật khẩu
    ↓
UserDetailsService       → lấy UserDetails từ DB
    ↓
UserDetails              → username, password mã hóa, roles
    ↓
SecurityContextHolder    → lưu Authentication đã xác thực cho thread hiện tại
```

Ví dụ thực tế: Nam nhập `nam / 123456` và bấm Login.

1. Filter bắt request, đóng gói thành token chưa xác thực.
2. Manager chuyển cho Provider.
3. Provider gọi `UserDetailsService` tìm `nam` trong DB, nhận về `UserDetails` có mật khẩu hash.
4. Provider so khớp `123456` với hash. Đúng thì tạo `Authentication` đã xác thực và lưu vào `SecurityContextHolder`. Sai thì báo lỗi `BadCredentialsException`.

6. HttpSecurity và SecurityFilterChain

`HttpSecurity` là đối tượng trung tâm để cấu hình bảo mật cho các HTTP request:

- **Request-based:** đặt quy tắc riêng cho từng URL/HTTP method (ví dụ `/admin` chỉ ADMIN, `/products` công khai).
- **Fluent API:** gọi nối chuỗi các phương thức để cấu hình phân quyền, cách đăng nhập (form, basic, OAuth2), quản lý session, xử lý exception.
- **SecurityFilterChain:** bean chứa chuỗi filter bảo mật áp dụng cho request.

Dependency:

```
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-security</artifactId>
</dependency>
```

Ví dụ cấu hình hoàn chỉnh:

```
@Configuration
@EnableWebSecurity
@EnableMethodSecurity   // để dùng @PreAuthorize
public class SecurityConfig {

      @Bean
    public PasswordEncoder passwordEncoder() {
        return new BCryptPasswordEncoder();
    }

    @Bean
    public SecurityFilterChain filterChain(HttpSecurity http) throws Exception {
        http
```

```
            .authorizeHttpRequests(auth -> auth
                .requestMatchers("/", "/home", "/css/**").permitAll()
                .requestMatchers("/admin/**").hasRole("ADMIN")
                .requestMatchers("/user/**").hasAnyRole("USER", "ADMIN")
                .anyRequest().authenticated()
            )
            .formLogin(form -> form
                .loginPage("/login")
                .defaultSuccessUrl("/home")
                .permitAll()
            )
            .logout(logout -> logout.logoutSuccessUrl("/login?logout"));

        return http.build();
    }

}
```

7. Các phương thức phân quyền

Phương thức	Ý nghĩa	Ví dụ
permitAll()	Ai cũng truy cập được, kể cả chưa đăng nhập	Trang chủ, trang login
denyAll()	Không ai được vào, kể cả admin	Khóa tạm một chức năng đang bảo trì
hasRole("X")	Phải có đúng vai trò X	/admin/** → hasRole("ADMIN")
hasAnyRole("X","Y")	Có ít nhất một trong các vai trò	/reports → hasAnyRole("ADMIN","MANAGER")
authenticated()	Chỉ cần đã đăng nhập, không quan tâm vai trò	Trang hồ sơ cá nhân

Lưu ý: `hasRole("ADMIN")` tự động thêm tiền tố `ROLE_`, nên trong DB/code bạn lưu là `ROLE_ADMIN`.

8. Các cách cung cấp thông tin đăng nhập

8.1. In-memory (lưu ngay trong code, dùng để test/demo)

```
@Bean
public UserDetailsService userDetailsService(PasswordEncoder encoder) {
    UserDetails user = User.withUsername("user")
            .password(encoder.encode("123"))
            .roles("USER")
            .build();

    UserDetails admin = User.withUsername("admin")
            .password(encoder.encode("admin123"))
            .roles("ADMIN", "USER")
            .build();

    return new InMemoryUserDetailsManager(user, admin);
}
```

`BCryptPasswordEncoder` băm mật khẩu một chiều. Ví dụ `123` thành `$2a$10$N9qo8uLOickgx2ZMRZoMye...`. Mỗi lần băm cho kết quả khác nhau nhưng vẫn kiểm tra khớp được. Đây là lý do không bao giờ lưu mật khẩu dạng thô.

8.2. JDBC (lấy từ cơ sở dữ liệu quan hệ, dùng trong thực tế)

Cách dùng bảng chuẩn của Spring:

```
@Bean
public UserDetailsService userDetailsService(DataSource dataSource) {
    return new JdbcUserDetailsManager(dataSource);
}
```

Bảng chuẩn gồm `users(username, password, enabled)` và `authorities(username, authority)`.

Hoặc dùng bảng tự thiết kế với truy vấn riêng:

```
@Bean
public UserDetailsManager users(DataSource dataSource) {
    JdbcUserDetailsManager manager = new JdbcUserDetailsManager(dataSource);
    manager.setUsersByUsernameQuery(
        "SELECT email, password, enabled FROM account WHERE email = ?");
    manager.setAuthoritiesByUsernameQuery(
        "SELECT email, role FROM account WHERE email = ?");
    return manager;
}
```

8.3. OAuth2 và LDAP

- **OAuth2:** đăng nhập qua Google/GitHub/Facebook, app chỉ nhận token xác thực.
- **LDAP:** xác thực qua máy chủ thư mục tập trung, phổ biến ở doanh nghiệp, trường học.

9. Các loại form đăng nhập

1. **Form của trình duyệt (HTTP Basic):** trình duyệt hiện hộp thoại popup xin username/password.
2. **Form mặc định của Spring Security:** trang `/login` tự sinh, không cần code HTML.
3. **Form tùy chỉnh:** bạn tự thiết kế giao diện.

```
.formLogin(form -> form.loginPage("/login").permitAll())  // dùng form tùy chỉnh
// hoặc
.httpBasic(Customizer.withDefaults())                      // dùng popup của trình duyệt
```

10. Demo từng bước

Bước 1: Thêm dependency

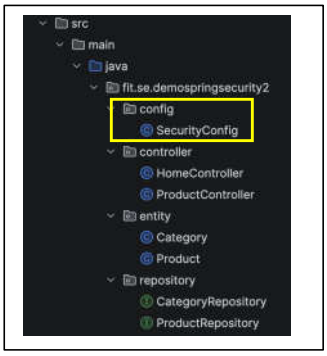
`spring-boot-starter-security`,

`thymeleaf-extras-springsecurity6 (thymeleaf)`

```
<dependency>
  <groupId>org.thymeleaf.extras</groupId>
  <artifactId>thymeleaf-extras-springsecurity6</artifactId>
</dependency>
```

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-security</artifactId>
</dependency>
```

Bước 2: Tạo package `config`, tạo class `SecurityConfig` với annotation `@Configuration`, `@EnableWebSecurity`.



Bước 3: Tạo bean `SecurityFilterChain` nhận tham số `HttpSecurity` (như ví dụ ở mục 6)

Bước 4: Ở Controller, dùng `@PreAuthorize` để phân quyền ngay trên method:

```
@Controller
public class ProductController {

    @GetMapping("/products")
    public String list() { return "products"; }          // ai đăng nhập cũng xem được

    @PreAuthorize("hasRole('ADMIN')")
    @PostMapping("/products/delete/{id}")
    public String delete(@PathVariable Long id) {        // chỉ ADMIN mới xóa được
        // xóa sản phẩm...
        return "redirect:/products";
    }
}
```

Bước 5: Ở View, kiểm tra ROLE để ẩn/hiện nút (dùng Thymeleaf + `thymeleaf-extras-springsecurity6`):

```
<a th:href="@{/products/add}" sec:authorize="hasRole('ADMIN')">Thêm sản phẩm</a>

<span sec:authorize="isAuthenticated()">
    Xin chào, <span sec:authentication="name"></span>
</span>
```

Với USER thường, nút "Thêm sản phẩm" sẽ không hiển thị. Nhưng đừng chỉ dựa vào việc ẩn nút, vẫn phải chặn ở server bằng `@PreAuthorize` hoặc `HttpSecurity`.

11. Các công nghệ nâng cao

Slide chỉ liệt kê tên để học tiếp:

- **OAuth2 (Single Sign-On):** đăng nhập một lần dùng cho nhiều hệ thống.
- **JWT (JSON Web Token):** token chứa thông tin user, thường dùng cho REST API không lưu session.
- **OpenID Connect:** lớp xác thực danh tính nằm trên OAuth2.

12. Tấn công CSRF

CSRF (Cross-Site Request Forgery) là kẻ tấn công lừa người dùng đang đăng nhập thực hiện hành động ngoài ý muốn, nhờ trình duyệt tự gửi kèm cookie/session.

Kịch bản tấn công bằng GET

Ngân hàng có API chuyển tiền: `GET http://bank.com/transfer?accountNo=1234&amount=100`

Kẻ tấn công gửi bạn email chứa link hấp dẫn:

```
<a href="http://bank.com/transfer?accountNo=5678&amount=1000">Xem ảnh mèo dễ thương</a>
```

Bạn đang đăng nhập bank.com ở tab khác, bấm vào link → trình duyệt gửi request kèm cookie của bạn → 1000 đồng chuyển vào tài khoản kẻ tấn công. Thậm chí không cần bấm, chỉ cần trang chứa thẻ ảnh là request tự chạy:

```
<img src="http://bank.com/transfer?accountNo=5678&amount=1000"/>
```

Kịch bản tấn công bằng POST

Dùng POST thì thẻ `<a>`/`<img>` không hoạt động, kẻ tấn công dùng form ẩn tự submit:

```
<body onload="document.forms[0].submit()">
  <form action="http://bank.com/transfer" method="POST">
    <input type="hidden" name="accountNo" value="5678"/>
    <input type="hidden" name="amount" value="1000"/>
  </form>
</body>
```

Mở trang này là tiền bay đi ngay. Nếu nạn nhân là admin, hậu quả có thể là mất cả hệ thống.

13. Phòng chống CSRF trong Spring

Nguyên lý: Spring sinh một **CSRF token** ngẫu nhiên gắn vào mỗi form. Mọi request làm thay đổi dữ liệu (POST, PUT, PATCH, DELETE) phải gửi kèm token này. Kẻ tấn công ở trang khác không thể biết token nên request giả mạo bị từ chối (lỗi 403).

- Từ Spring Security 4.x, CSRF bật mặc định.
- Token nằm trong attribute `_csrf` gồm: `token`, `parameterName`, `headerName`.
- GET không cần token vì GET chỉ nên dùng để đọc, không thay đổi dữ liệu.

Tắt CSRF (không khuyến khích)

Slide ghi kiểu cũ `http.csrf().disable()`. Từ Spring Security 6 phải viết:

```
http.csrf(csrf -> csrf.disable());
```

Chỉ nên tắt khi làm REST API stateless dùng JWT.

Form HTML (Thymeleaf)

Thymeleaf tự thêm token nếu dùng `th:action`:

```
<form th:action="@{/transfer}" method="post">
    <input type="text" name="accountNo"/>
    <input type="number" name="amount"/>
    <button type="submit">Chuyển tiền</button>
</form>
```

Với JSP hoặc HTML thuần, thêm thủ công (như Slide 25):

```
<input type="hidden" name="${_csrf.parameterName}" value="${_csrf.token}"/>
```

Gọi AJAX/JSON (jQuery)

```
<!-- 1. Đặt token vào meta tag -->
<meta name="_csrf" content="${_csrf.token}"/>
<meta name="_csrf_header" content="${_csrf.headerName}"/>

<script>
// 2. Đọc giá trị từ meta
var token  = $("meta[name='_csrf']").attr("content");
var header = $("meta[name='_csrf_header']").attr("content");

// 3. Gắn vào header của mọi request AJAX
$(document).ajaxSend(function (e, xhr, options) {
    xhr.setRequestHeader(header, token);
});
</script>
```

Tóm tắt:

1. **Authentication** = bạn là ai, **Authorization** = bạn được làm gì.
2. Luồng: Filter → Manager → Provider → UserDetailsService → UserDetails → SecurityContextHolder.
3. Cấu hình qua bean `SecurityFilterChain` + `HttpSecurity`; phân quyền bằng `permitAll/hasRole/authenticated...` hoặc `@PreAuthorize`.
4. Lưu user: in-memory (test), JDBC (thực tế), OAuth2/LDAP (nâng cao); luôn mã hóa mật khẩu bằng BCrypt.
5. CSRF được chặn bằng token; mặc định bật, chỉ gửi kèm với POST/PUT/PATCH/DELETE.

Yêu cầu: Tạo table user (id,usename,password,role) trong database thực hiện xác thực user cho project với 3 role GUEST, USER, ADMIN tương ứng với từng các chức năng của project



123 id	A-Z username	A-Z password	A-Z role
1	admin1	123	ADMIN
2	admin2	123	ADMIN
3	user1	111	USER
4	user2	222	USER
5	guest1	333	GUEST
6	guest2	444	GUEST

Chức năng	GUEST	USER	ADMIN
Xem danh sách, tìm theo id, name, position	✓	✓	✓
Tìm theo salary (dữ liệu nhạy cảm)	✗	✓	✓
Thêm Employee	✗	✓	✓
Sửa Employee	✗	✓	✓
Xóa Employee	✗	✗	✓

Các URL tương ứng cho các chức năng ứng với mỗi method trong controller

URL	Method	Chức năng	Role
`/employees`	GET	Danh sách	ADMIN, USER, GUEST
`/employees/search`	GET	Tìm kiếm	ADMIN, USER, GUEST
`/employees/{id}`	GET	Chi tiết	ADMIN, USER, GUEST
`/employees/new`	GET	Form thêm	ADMIN, USER
`/employees`	POST	Thêm	ADMIN, USER
`/employees/{id}/edit`	GET	Form sửa	ADMIN, USER
`/employees/{id}`	POST	Cập nhật	ADMIN, USER
`/employees/{id}/delete`	POST	Xóa	ADMIN